---

title: “Implementation defined coroutine extensions”
document: P3203
date: 2024-03-22
audience: Core
author:

- name: Klemens David Morgenstern
  email: klemens.d.morgenstern@gmail.com

Proposed Changes

This paper proposes two wording changes to the standard that would make it legal (i.e. implementation defined) for users to provide their own coroutine implementations.

coroutine.handle.general-2

> If a program declares an explicit or partial specialization of coroutine_handle, the behavior is undefined.

Changed to

> If a program declares an explicit or partial specialization of coroutine_handle, the behavior is **implementation defined**.

coroutine.handle.export.import-2

> *Preconditions*: addr was obtained via a prior call to address on an object whose type is a specialization of coroutine_handle.

Changed to

> *Preconditions*: addr was obtained via a prior call to address on an object whose type is a specialization of coroutine_handle
> **which is neither explicit nor partial, obtained by a call to address on noop_coroutine_handle or points to a section of memory that is ABI compatible with the implementation provided by the former**.

Technical background

The coroutine frame implementations are the same on MSVC, Gcc & Clang and look like this for a given `promise_type`.

```
struct coroutine_frame
{
  void (resume *) (coroutine_frame * );
  void (destroy *)(coroutine_frame * );
  promise_type promise;
  // auxiliary data goes here, like the function arguments
};
```

The `std::coroutine_handle` functions to `resume` & `destroy` call the appropriate function pointers, whereas `promise` returns a reference to the `promise` member and `done` checks if `resume` is `null`.

Motivation

Allowing users to provide their own coroutine types is useful for public interfaces.

An example can be found in boost.cobalt where `python` awaits `C++` coroutines.
Because this example does not include defined behaviour, it uses a superfluous coroutine `py_coroutine` as glue,
which causes an additional & unnecessary allocation & indirection.
This superfluous coroutine could be eliminated with the proposed changed, which is likely even more useful for bindings to faster languages like `rust`.

Stackful coroutines

Boost.cobalt also has an experimental implementation that provides stackful coroutines
as an alternative runner for C++20 coroutines.

That is, instead of

```
boost::cobalt::promise<void> stackless()
{
  co_await boost::asio::post(boost::cobalt::use_op); // the simplest possible async operation
}

boost::cobalt::promise<void> cs = stackless();
```

it can be run stackful (powered by boost.context) with the following code:

```
boost::cobalt::promise<void> stackful(
    boost::cobalt::experimental::context<boost::cobalt::promise<void>> ctx)
{
  ctx.await(boost::asio::post(boost::cobalt::use_op));
}

boost::cobalt::promise<void> cs = boost::cobalt::experimental::make_context(&stackful);
```

The `coroutine_frame` gets created in `make_context` and embedded in the coroutine stack, avoiding a second allocation.
This gives a user the benefits of a stackful coroutine (like interacting with coroutine unaware APIs) while being able
to interact with any `co_await`-able API (such as boost.cobalt's utilities) without any overhead.

It is worth nothing, that also (already) works with `ucontext` and `WinFiber`,
since `boost.context` supports either.

Any asynchronous completion

Asynchronous completion has been a hotly debated issue over the last few years with may papers involved.
By allowing user extensions here, any completion could be plugged into a coroutine_handle.
If we are furthermore allowed to specialize these handles, the overhead can be minimized by templating the `await_suspend` function on an awaitable.

```
struct my_awaitable
{
    bool await_ready();
    template<typename Promise>
    void await_suspend(std::coroutine_promise<Promise> h); // this makes it transparent to the compiler
    void await_resume();
};
```

Conclusion

This relatively minor change is purely legal, as it only declares currently undefined behaviour as implementation defined behaviour.
That is, no work of any compiler vendor is needed.

These changes will allow libraries like boost.cobalt, which shares the author with this paper,
to experiment and provide more functionality and integration into existing code bases that do not run on C++20 coroutines yet.

It furthermore opens up the only model for any asynchronous completion. This might not be the most efficient model, but it will allow developers to provide public APIs that can be consumed by other things than coroutines.

The main feature however will be that other coroutine implementations, such as fibers, or models from other languages.